ESCUELA DE INGENIERÍA EN CIENCIAS Y SISTEMAS
FACULTAD DE INGENIERÍA
UNIVERSIDAD DE SAN CARLOS DE GUATEMALA

MANUAL USUARIO

PROYECTO FINAL LFP

Nombre: Bily Estuardo vallecidos folgar

Curso: Lenguajes Formales y Programación

Fecha: 01/05/2026

INTRODUCCIÓN

El presente manual tiene como objetivo orientar al usuario en el manejo y funcionamiento de **TaskScript**, una herramienta integral diseñada para la gestión y organización de proyectos mediante la metodología **Kanban**. En el entorno actual de desarrollo de software, la visualización clara de las tareas es fundamental para el éxito de cualquier equipo, y TaskScript surge como una solución técnica que combina la robustez del procesamiento de lenguajes formales con una interfaz moderna y accesible.

Este sistema permite procesar archivos de configuración con extensión .task, los cuales contienen la estructura jerárquica de tableros, columnas y tareas. A través de un motor de análisis desarrollado en **C++** y una interfaz interactiva en **React**, el usuario podrá transformar definiciones de texto plano en reportes visuales detallados, facilitando el seguimiento de actividades, la detección de errores sintácticos y la administración eficiente de flujos de trabajo.

A lo largo de este documento, se detallarán los requerimientos mínimos de instalación, los pasos para la carga de archivos y la interpretación de los distintos reportes generados, asegurando que el usuario pueda aprovechar al máximo todas las funcionalidades de la aplicación.

1. **Compilación e Instalación de Dependencias**

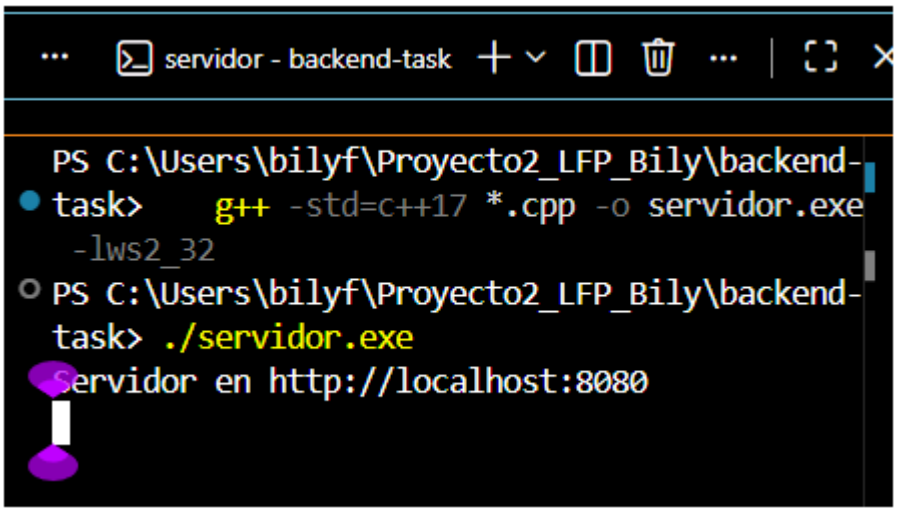
   El sistema de TaskScript Pro funciona bajo una arquitectura separada, lo que significa que el motor de análisis y la interfaz gráfica deben iniciarse de forma independiente. Siga estos pasos detallados para ejecutar la herramienta:

   A. Servidor de Compilación (Backend C++) Este módulo se encarga de procesar el texto y aplicar las reglas gramaticales.

   1. Abra una terminal de comandos y navegue hasta la ruta /backend/src de su proyecto.

   2. Compile el código fuente ejecutando la siguiente instrucción (asegúrese de tener MinGW instalado): g++ server.cpp LexicalAnalyzer.cpp SyntaxAnalyzer.cpp ReportGenerator.cpp Token.cpp ErrorManager.cpp -o server.exe -lws2_32

2. Inicie el servidor ejecutando el comando ./server.exe. La terminal le confirmará que el servidor está a la escucha en el puerto local 8080.



B. Interfaz de Usuario (Frontend React) Este módulo provee la interfaz visual para interactuar con el sistema.

1. Abra una nueva terminal (sin cerrar la del servidor) y navegue a la carpeta /frontend.

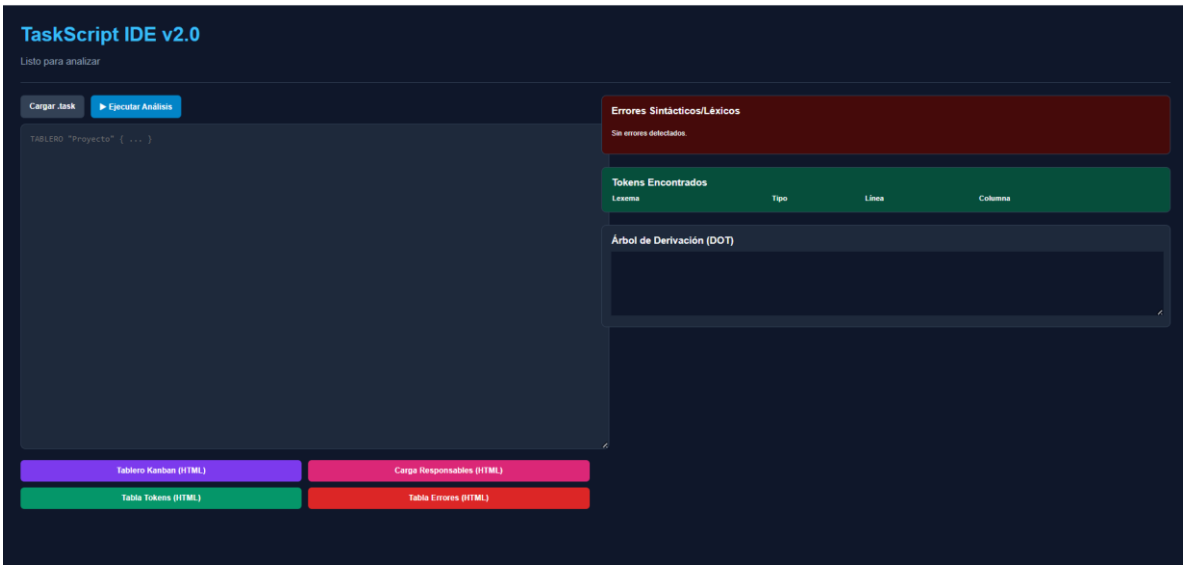
2. Instale las dependencias necesarias de Node.js ejecutando: npm install

3. Levante la aplicación ejecutando el comando: npm run dev

4. Abra su navegador web e ingrese al enlace local proporcionado por la terminal (generalmente http://localhost:5173)

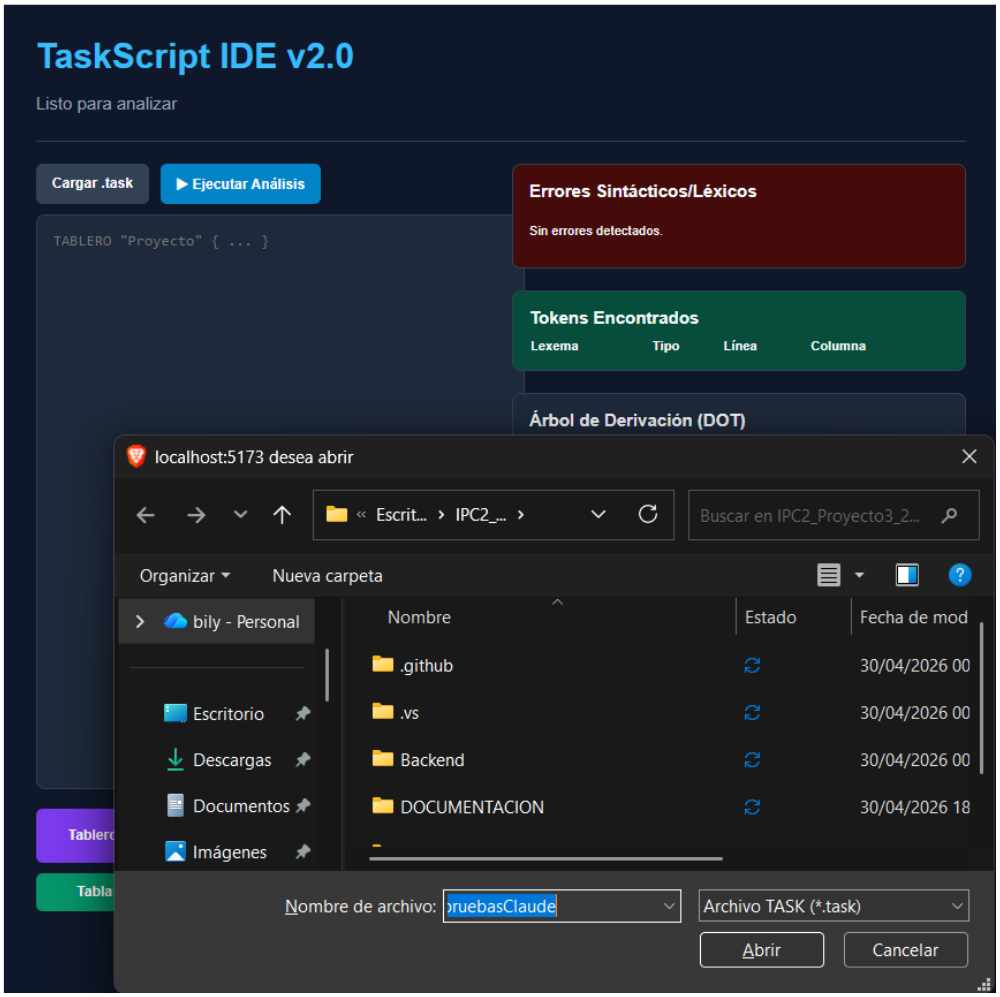
```
PS C:\Users\bilyf\Proyecto2_LFP_Binpm
run devcto-task>

> mi-proyecto-task@0.0.0 dev
> vite

11:21:18 p. m. [vite] (client) Re-opti
mizing dependencies because vite confi
g has changed

  VITE v8.0.10  ready in 383 ms

  → Local:   http://localhost:5173/
  → Network: use --host to expose
  → press h + enter to show help
```

3. Carga y Análisis de Archivos en la GUI
La interfaz gráfica principal está dividida en dos paneles funcionales para facilitar el flujo de trabajo.

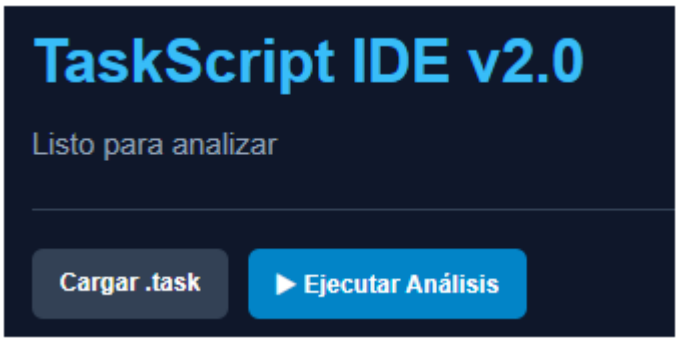


Para procesar un documento, siga este flujo: 1. Cargar Archivo: En el panel izquierdo, haga clic en el botón gris "Cargar Archivo". Se abrirá el explorador de su sistema operativo; seleccione su documento con extensión .task.

2. Editor de Texto: El contenido del archivo se volcará en el área de texto central. Puede modificar libremente el código aquí antes de enviarlo.



Analizar Código: Una vez que el código esté listo, haga clic en el botón azul "Analizar Código". La herramienta enviará la información al servidor y en instantes el panel derecho se actualizará con los resultados.



4. Estructura del Archivo .task
   El motor de TaskScript exige que la información esté jerarquizada correctamente. Todo archivo debe iniciar con un TABLERO, contener una o más secciones de COLUMNA, y dentro de estas, declarar las tareas. A continuación, se presentan dos ejemplos completos de uso:
   Ejemplo 1: Tablero Básico Ideal para organizar proyectos rápidos y lineales:

```
TABLERO "Fase Inicial" {
    COLUMNA "Pendientes" {
        tarea: "Configurar repositorio" [prioridad: ALTA, responsable: "Jorge",
fecha_limite: 2026-05-01]
    };
};
```

   Ejemplo 2: Tablero Multicolumna Estructura avanzada para un flujo de trabajo ágil completo, mostrando múltiples tareas y atributos:

```
TABLERO "Proyecto LFP" {
    COLUMNA "Por Hacer" {
        tarea: "Diseñar AFD" [prioridad: ALTA, responsable: "Jorge",
fecha_limite: 2026-05-01],
        tarea: "Implementar Lexer" [prioridad: MEDIA, responsable: "Maria",
fecha_limite: 2026-05-08]
    };
    COLUMNA "Completado" {
        tarea: "Investigar Qt" [prioridad: BAJA, responsable: "Pedro",
fecha_limite: 2026-04-20]
    };
};
```

5. Interpretación de la Tabla de Tokens y Mensajes de Error
   Una vez presionado el botón de análisis, el panel derecho desplegará dos cuadros de diagnóstico para evaluar la salud de su código:

Tabla de Tokens Reconocidos (Verde):
Muestra el desglose de cada unidad de texto que el sistema procesó correctamente. Cada fila le indicará el "Lexema" exacto, el tipo de dato detectado (palabra reservada, cadena, fecha, etc.) y su posición precisa mediante número de línea y columna.

Tabla de Errores Detectados (Roja): Enumera las anomalías encontradas durante la lectura.
Errores Léxicos: Se muestran cuando ingresa caracteres ajenos al lenguaje (como @ o #). El sistema los registra, los ignora y continúa evaluando el resto del archivo para que usted pueda generar los reportes de igual forma. o Errores Sintácticos: Aparecen cuando la estructura está mal formada (como omitir una coma o un corchete de cierre). En este escenario, la tabla le indicará qué elemento se esperaba recibir. Por medidas de seguridad, si existe un error

sintáctico, la generación de reportes HTML se bloqueará automáticamente.

5. Navegación de Reportes HTML y Árbol DOT Si su código es estructuralmente válido, la aplicación habilitará cuatro botones en la esquina inferior izquierda. Al hacer clic en ellos, se abrirán en nuevas pestañas los siguientes elementos:

A. Reporte de Tokens

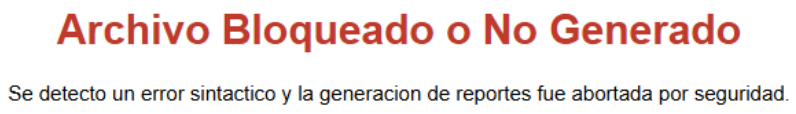
Tabla de Tokens

No.	Lexema	Tipo	Línea	Columna
1	TABLERO	KW_TABLERO	1	1
2	"Sistema de Ventas Q2-2026"	LIT_CADENA	1	9
3	{	LLAVE_ABRE	1	37
4	COLUMNA	KW_COLUMNA	2	5
5	"Backlog"	LIT_CADENA	2	13
6	{	LLAVE_ABRE	2	23
7	tarea	KW_TAREA	3	9
8	:	DOS_PUNTOS	3	14
9	"Analizar requerimientos del cliente"	LIT_CADENA	3	16
10	[	CORCHETE_ABRE	3	54
11	prioridad	KW_PRIORIDAD	3	55
12	:	DOS_PUNTOS	3	64
13	ALTA	VAL_ALTA	3	66
14	,	COMA	3	70
15	responsable	KW_RESPONSABLE	3	72
16	:	DOS_PUNTOS	3	83
17	"Andrea"	LIT_CADENA	3	85
18	,	COMA	3	93
19	fecha_limite	KW_FECHA_LIMITE	3	95
20	:	DOS_PUNTOS	3	107
21	2026-05-15	LIT_FECHA	3	109
22	]	CORCHETE_CIERRA	3	119
23	,	COMA	3	120

Al presionar el botón verde "Ver Reporte de Tokens", se abrirá un documento HTML con una tabla auditable de todos los elementos léxicos válidos que el sistema reconoció. Este reporte es ideal para verificar paso a

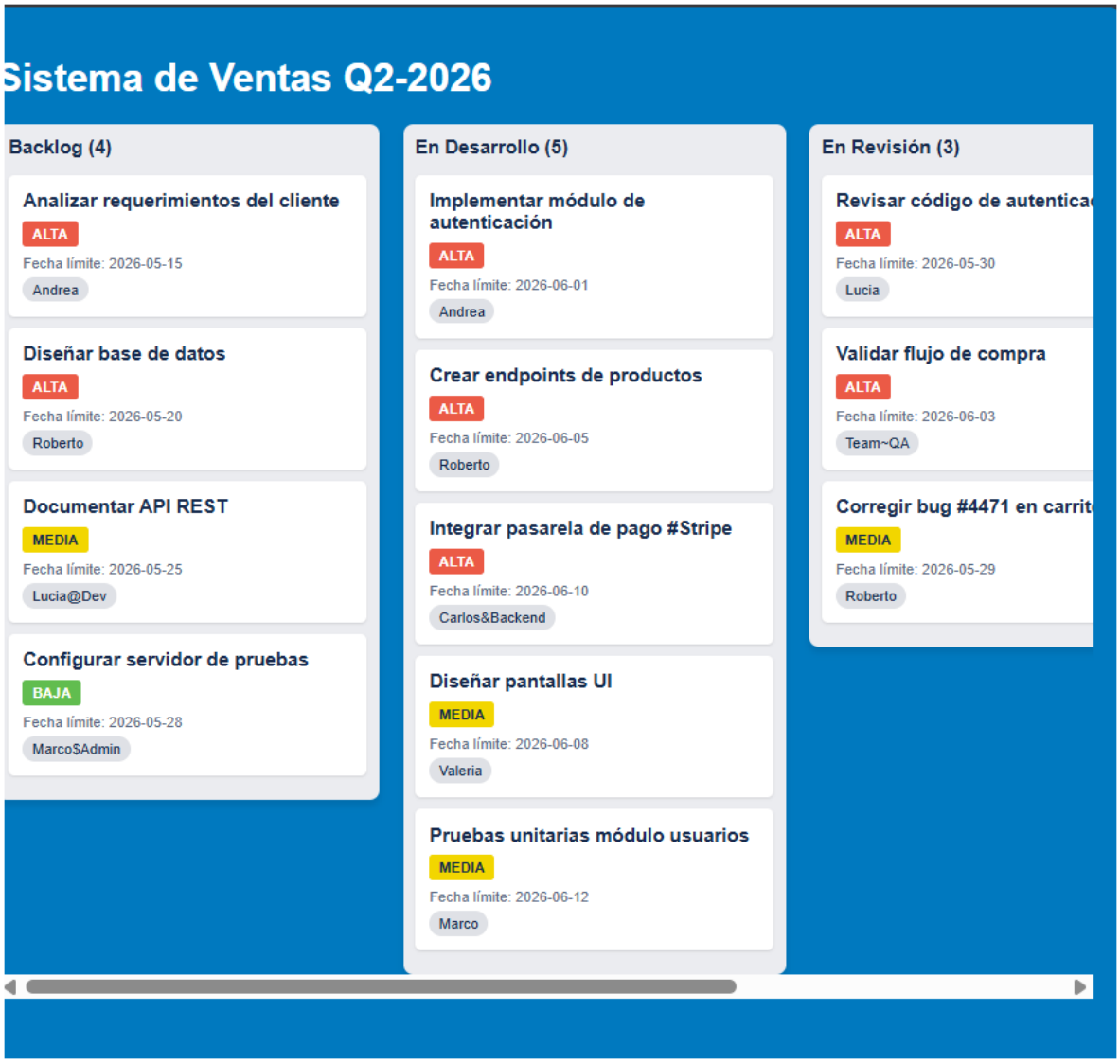
paso cómo el autómata interpretó cada palabra, número y símbolo del archivo fuente, mostrando su tipo y ubicación exacta.

B. Reporte de Errores



Al presionar el botón rojo "Ver Reporte de Errores", se generará un documento oficial con el listado de anomalías detectadas. A diferencia de la vista rápida en la interfaz, este reporte sirve como una bitácora formal de fallos (léxicos o sintácticos) que el equipo de desarrollo puede guardar o imprimir para corregir la sintaxis del proyecto posteriormente. Si el código es perfecto, el reporte mostrará un mensaje de análisis limpio.

C. Tablero Kanban Visual



Al presionar el botón morado, se generará una pizarra visual del proyecto. Cada columna declarada agrupará sus tareas en formato de tarjetas, empleando un código de colores (Rojo, Amarillo y Verde) para identificar rápidamente el nivel de prioridad de cada asignación. (Nota: Este reporte solo se genera si no hay errores sintácticos).

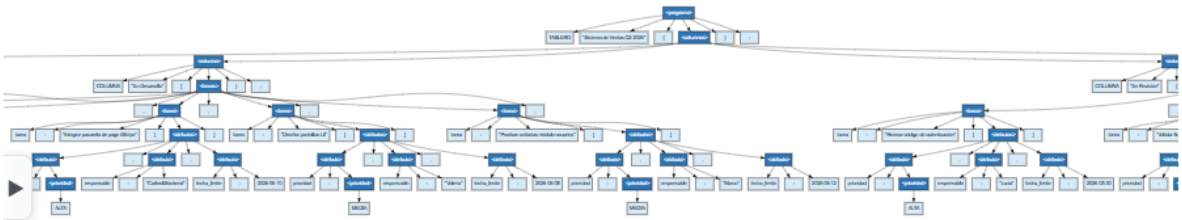
D. Carga por Responsable

Carga por Responsable

Responsable	Tareas Asignadas	Alta	Media	Baja	Distribución
Andrea	3	3	0	0	18%
Carlos	1	0	0	1	6%
Carlos&Backend	1	1	0	0	6%
Carlos^Ops	1	0	1	0	6%
Lucia	1	1	0	0	6%
Lucia@Dev	1	0	1	0	6%
Marco	2	0	1	1	12%
Marco$Admin	1	0	0	1	6%
Roberto	3	2	1	0	18%
Team~QA	1	1	0	0	6%
Valeria	1	0	1	0	6%

Este botón naranja despliega una tabla estadística interactiva. Agrupa las asignaciones por integrante del equipo y proporciona un desglose numérico de cuántas tareas (y de qué prioridad) le corresponden a cada uno. Incluye una barra de distribución que grafica el porcentaje de carga laboral individual sobre el total del proyecto. (Nota: Este reporte solo se genera si no hay errores sintácticos).

E. Código Árbol DOT (Graphviz)



En el recuadro oscuro ubicado en el extremo inferior del panel derecho, se imprime el código fuente en formato DOT. Este código representa el Árbol de Sintaxis Abstracta (AST) generado por el compilador. Para visualizar el diagrama jerárquico, solo debe copiar la totalidad del texto y pegarlo en cualquier herramienta visualizadora de Graphviz en línea.

CONCLUSIÓN

La implementación de **TaskScript** representa una solución integral y eficiente para la gestión de proyectos bajo la metodología Kanban, logrando un equilibrio óptimo entre la rigurosidad técnica de los lenguajes formales y la practicidad de las interfaces de usuario modernas. A través de este sistema, se ha demostrado que la automatización del flujo de trabajo no tiene por qué ser compleja para el usuario final; por el contrario, el software abstrae la complejidad del análisis sintáctico y léxico para entregar resultados visuales, claros y accionables.

Una de las mayores fortalezas de esta herramienta es su capacidad de **retroalimentación inmediata**. El usuario no solo obtiene un tablero organizado, sino que cuenta con un sistema de diagnóstico que identifica errores en el archivo de entrada con precisión de fila y columna. Esto convierte a TaskScript en una herramienta educativa y operativa que minimiza la frustración del usuario ante archivos mal formados, garantizando que la integridad de los datos se mantenga en todo momento.

Finalmente, el éxito de esta aplicación radica en su **escalabilidad y portabilidad**. Al generar reportes en formato HTML, se asegura que el tablero de tareas pueda ser compartido y visualizado en cualquier plataforma sin dependencia de software propietario. TaskScript no es solo un proyecto académico; es un prototipo funcional que demuestra cómo la teoría de autómatas y gramáticas, cuando se combina con el desarrollo web contemporáneo, puede producir software de alta calidad, intuitivo y alineado con las necesidades de organización del mundo real.

Con el uso correcto de este manual, el usuario queda plenamente facultado para dominar la sintaxis del lenguaje y transformar sus definiciones de texto en potentes herramientas de visualización, optimizando así la productividad y el control sobre sus proyectos más importantes.